

16_++고급기능-2

2101080 정진용

1.



실습과제 1

18

- 아래는 `std::bind` 함수의 선언이다. 여기서 매개변수 선언에서 `&&`의 의미를 설명하라.

```
template< class F, class... Args >  
/*unspecified*/ bind(F&& f, Args&&... args);
```

(ANSWER)

1. 우측값(R-value) 참조 : `&&`는 기본적으로 임시 객체나 리터럴 상수와 같은 "우측값"만 참조할 수 있는 변수를 선언할 때 사용됩니다. 예를 들어 `int&& r = 10;`은 가능하지만, 변수를 참조하는 `int i = 5; int&& r = i;`는 컴파일 오류가 발생합니다. 이는 임시 객체의 소유권을 "이동(move)"하여 불필요한 복사를 피하고 성능을 향상시키는 이동 의미론(Move Semantics)의 핵심 요소입니다.
2. 전달 참조(Forwarding Reference): `std::bind`의 선언에서처럼 `F&& f`와 `Args&&... args`는 템플릿 타입 추론과 결합되어 전달 참조로 동작합니다.
 - 함수에 좌측값(L-value, 변수 등)이 전달되면, 해당 매개변수는 좌측값 참조(`&`) 타입으로 추론됩니다.
 - 함수에 우측값(R-value, 상수, 임시 객체 등)이 전달되면, 해당 매개변수는 우측값 참조(`&&`) 타입으로 추론됩니다.

결론적으로, `std::bind` 선언에서 `&&`는 함수 `f`와 인자 `args`가 좌측값이든 우측값이든 상관없이 모두 받을 수 있게 해주는 특별한 문법입니다. 이를 통해 `std::bind`는 전달받은 인자들을 원래의 값 속성(좌측값 또는 우측값) 그대로 내부 함수에 전달(perfect forwarding)하여, 불필요한 복사를 방지하고 효율성을 극대화할 수 있습니다.

2.

실습과제 2

- 인터넷에서 함수의 인자로 `std::bind` 함수를 사용한 예제를 찾아서 실행해보고 소스코드를 자세히 설명하시오.

코드.

```
#include <iostream>
#include <vector>
#include <functional>
#include <algorithm>

// 숫자가 특정 임계값보다 큰지 확인하는 일반 함수
bool isGreaterThan(int value, int threshold) {
    return value > threshold;
}

int main() {
    // 테스트용 정수 벡터 생성
    std::vector<int> numbers = {10, 55, 23, 8, 91, 44, 78};

    // std::bind를 사용하여 isGreaterThan 함수의 두 번째 인자를 50으로 고정
    // placeholders::_1은 count_if가 전달할 첫 번째 인자(벡터의 각 요소)를 의미
    auto isGreaterThan50 = std::bind(isGreaterThan, std::placeholders::_1, 50);

    // std::count_if는 세 번째 인자로 받은 함수(isGreaterThan50)를 벡터의
    // 모든 요소에 대해 호출하고, 그 결과가 true인 요소의 개수를 반환
    int count = std::count_if(numbers.begin(), numbers.end(), isGreaterThan50);

    // 결과 출력
    std::cout << "50보다 큰 숫자의 개수: " << count << std::endl;

    return 0;
}
```

콘솔.

```
Microsoft Visual Studio 디버그 × + ▾
Func1 호출! hello
Func2 호출! c
Func3 호출!
C:\Users\spick\source\repos\opencv\x64\Debug\vvvvvvvvvv.exe(프로세스 25736)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

3.

실습과제 3

- 인터넷에서 `std::chrono_literals` 을 사용한 예제를 찾아서 실행해보고 소스코드를 자세히 설명하시오.

(ANSWER)

```
#include <iostream>
#include <chrono>
#include <thread>

// using namespace std::chrono_literals 지시문을 사용하여
// ms, s 등의 시간 리터럴을 바로 사용할 수 있도록 함
using namespace std::chrono_literals;

int main() {
    // 1. 밀리초(ms) 리터럴 사용
    std::cout << "1. 500ms 동안 대기합니다..." << std::endl;
    // 500ms는 operator""ms(500)으로 변환되어 std::chrono::milliseconds 객체를 생성
    std::this_thread::sleep_for(500ms);
    std::cout << "    대기 완료!" << std::endl;

    // 2. 초(s) 리터럴 사용
    std::cout << "2. 2초 동안 대기합니다..." << std::endl;
    // 2s는 operator""s(2)로 변환되어 std::chrono::seconds 객체를 생성
    std::this_thread::sleep_for(2s);
    std::cout << "    대기 완료!" << std::endl;

    // 3. 리터럴을 사용한 시간 계산
    auto total_sleep_time = 1s + 250ms;
    std::cout << "3. 총 " << total_sleep_time.count() << "밀리초 동안 대기합니다..." <<
std::endl;
    std::this_thread::sleep_for(total_sleep_time);
    std::cout << "    대기 완료!" << std::endl;
```

```
return 0;
```

```
}
```

4.

실습과제 4

- 23페이지 예제처럼 인터넷에서 `std::function` 클래스를 함수에 매개 변수에 사용한 예제를 찾아서 실행해보고 소스코드를 자세히 설명 하시오.

코드.

```
#include <iostream>
#include <functional> // std::function을 사용하기 위해 반드시 포함해야 합니다. [cite: 223]
#include <string>

// 계산을 수행하고 결과를 출력하는 고차 함수(Higher-order function)
// 세 번째 인자로 어떠한 연산을 수행할지 'std::function' 객체로 전달받습니다.
void calculate(int a, int b, std::function<int(int, int)> operation) {
    int result = operation(a, b); // 전달받은 연산(operation)을 실행합니다.
    std::cout << "결과: " << result << std::endl;
}

// 1. 일반 함수: 덧셈
int add(int a, int b) {
    return a + b;
}

// 2. 일반 함수: 뺄셈
int subtract(int a, int b) {
    return a - b;
}

int main() {
    int a = 10, b = 5;

    std::cout << a << " + " << b << " 연산" << std::endl;
    calculate(a, b, add); // 'add' 함수를 인자로 전달

    std::cout << a << " - " << b << " 연산" << std::endl;
    calculate(a, b, subtract); // 'subtract' 함수를 인자로 전달

    std::cout << a << " * " << b << " 연산" << std::endl;
    // 3. 람다 표현식: 곱셈
    calculate(a, b, [](int x, int y) { return x * y; }); // 곱셈 람다를 인자로 전달

    return 0;
}
```

콘솔.

```
Microsoft Visual Studio 디버그 × + -
10 + 5 연산
결과: 15
10 - 5 연산
결과: 5
10 * 5 연산
결과: 50
C:\Users\spick\source\repos\opencv\x64\Debug\vvvvvvvvvv.exe(프로세스 12404)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

`calculate` 함수는 두 정수와 함께 `std::function<int(int, int)>` 타입의 `operation` 객체를 매개변수로 받는데, 이 `operation` 객체는 두 개의 `int`를 인자로 받아 `int`를 반환하는 모든 호출 가능한 것을 담을 수 있습니다. 따라서 `main` 함수에서는 미리 정의된 `add`와 같은 일반 함수 포인터를 전달할 수도 있고, 곱셈 연산처럼 별도의 함수 정의 없이 `[](int x, int y){ return x*y; }`와 같은 람다 표현식을 즉석에서 작성하여 전달할 수도 있습니다. `calculate` 함수는 내부적으로 `operation(a, b)` 코드를 통해 어떤 기능이 전달되었든 동일한 방식으로 실행하여 결과를 얻으며, 이처럼 `std::function`을 통해 수행할 연산 자체를 파라미터로 주입하는 방식은 단 하나의 함수를 매우 유

연하고 확장성 있게 만드는 좋은 방법을 보여줍니다.